

Enterprise Insurance Fraud Detection Platform

Neo4j Desktop Architecture & Developer Implementation Specification

Version: 1.0

Date: May 2026

Primary Graph Platform: Neo4j Desktop

Deployment Model: Local Development / Enterprise Simulation

IMPORTANT IMPLEMENTATION CONSTRAINT

This project **MUST** use:

- Neo4j Desktop **ONLY**

The implementation **MUST NOT**:

- Depend on Neo4j AuraDB
- Use Aura-specific features
- Require cloud deployment
- Assume managed Neo4j services

The solution **SHOULD**:

- Remain migration-ready for future AuraDB deployment
 - Be modular and portable
 - Support future enterprise scaling
-

1. Project Objective

Design and implement a production-style enterprise insurance fraud detection platform capable of:

- Real-time fraud scoring
- Batch fraud analytics
- Fraud ring detection
- Cross-system identity resolution
- Cross-carrier fraud simulation
- Temporal fraud analysis

- Multi-modal AI integration
- Investigator workflows
- Graph analytics using Neo4j Desktop

The system shall simulate a real-world East Coast US insurance carrier ecosystem centered around Hartford, Connecticut regional insurance operations.

2. Supported Insurance Domains

Property & Casualty

- Property
- Liability
- Fleet Auto
- Umbrella
- Flood
- Cyber Liability
- EPL
- D&O
- Professional Liability

Workers Compensation

- Workers Compensation
- Disability

Benefits & Life

- Key Person Life Insurance
 - Employee Benefits
 - HRA
 - HSA
-

3. Scale Requirements

Target Simulation

- 100K claims/year
- 10 years historical data
- ~1M claims simulated

Graph Size Expectations

Expected eventual scale:

- 8M–15M nodes
- 15M–40M relationships

Neo4j Desktop Considerations

Developer must:

- Tune JVM memory settings
 - Use indexes aggressively
 - Use constraints on all primary identifiers
 - Use MERGE instead of CREATE
 - Batch-load data using UNWIND
 - Avoid Cartesian products
 - Use APOC procedures where appropriate
-

4. Enterprise Architecture

Layer 1 — Data Sources

Internal Systems

System	Purpose
Guidewire	Claims
Duck Creek	Policy Admin
TriZetto	Medical Billing
SAP	Finance
Salesforce	CRM
Workday	HR/Benefits
Sedgwick	WC TPA

External Systems

System	Purpose
ISO ClaimSearch	Cross-carrier fraud
LexisNexis	Identity verification
NICB	VIN theft/fraud
CMS	Provider exclusions

System	Purpose
NOAA	Weather validation
DMV feeds	Registration validation
OFAC	Compliance
Court records	Litigation history
IRS TIN matching	Provider/vendor validation

Layer 2 — Ingestion Pipeline

Requirements

Support:

- Real-time ingestion simulation
- Batch ingestion
- Idempotent loading
- Replay-safe ingestion
- Deduplication

Initial Stack

- Python ingestion services
- CSV/JSON ingestion
- Neo4j Bolt Driver
- Kafka-ready architecture (future)

Event Topics

- claims.fnol
 - claims.update
 - medical.bill
 - repair.invoice
 - payment.issued
 - policy.change
 - external.enrichment
-

Layer 3 — Multi-Modal AI Processing

Computer Vision Targets

- Auto damage photos
- Property damage photos

- Repair invoices
- Accident scene photos
- Medical documentation

Five-Layer Forensic Pipeline

1. Metadata & EXIF Analysis
2. Pixel Forensics (ELA/FFT)
3. Duplicate Image Detection
4. Cross-Document Consistency
5. Temporal Consistency

OCR + LLM Requirements

The platform shall:

- Extract entities automatically
 - Load graph entities automatically
 - Detect repeated templates
 - Detect anomalous narratives
 - Detect document inconsistencies
-

Layer 4 — Entity Resolution

Requirements

Support:

- Cross-system identity resolution
- Fuzzy matching
- Cross-carrier linking
- Master entity creation

Matching Signals

- Name
 - Phone
 - Address
 - DOB
 - SSN last4
 - Device fingerprint
 - Email hash
-

Layer 5 — Graph Store

Database Platform

- Neo4j Desktop

Required Neo4j Plugins

- APOC
- Graph Data Science (GDS)

Graph Design Principles

- MERGE-only ingestion
 - Strong constraints
 - Temporal indexing
 - Idempotent loads
 - No duplicate generation
 - Config-driven scoring
-

Layer 6 — Intelligence Engine

Graph Algorithms

- Connected Components
- Louvain Community Detection
- PageRank
- Betweenness Centrality
- Weakly Connected Components
- Temporal graph analytics

Fraud Scoring

Scoring must:

- Be tunable
- Be YAML-driven
- Support LOB-specific weights
- Support explainable scoring
- Support payment hold logic

Configuration Requirement

A single master YAML configuration file shall store all fraud scoring weights.

Example:

```
auto:
  staged_accident_weight: 35
  duplicate_claim_weight: 20

workers_comp:
  ime_collusion_weight: 45
```

The scoring engine must reload weights without code changes.

Layer 7 — Agentic AI

Operating Model

Human-in-the-loop co-pilot.

AI Responsibilities

- Assemble evidence
- Suggest next actions
- Draft SIU narratives
- Build timelines
- Recommend referrals

Human Responsibilities

- Approve investigations
 - Override AI decisions
 - Add evidence
 - Close cases
-

Layer 8 — Investigator Dashboard

Dashboard Views

- Graph View
- Temporal Timeline
- Geospatial View
- Case Queue
- Cross-carrier view

Recommended Free Tools

Tool	Purpose
Neo4j Bloom	Graph exploration

Tool	Purpose
vis.js	Graph UI
Leaflet.js	Geospatial maps
Plotly	Timelines/charts

5. Core Graph Schema

Node Labels

1. Claim
 2. ClaimVersion
 3. CoverageLine
 4. ClaimPayment
 5. Person
 6. Employee
 7. Dependent
 8. Policy
 9. Vehicle
 10. Fleet
 11. MedicalProvider
 12. MedicalBill
 13. RepairShop
 14. Attorney
 15. TowCompany
 16. Employer
 17. Address
 18. Phone
 19. CrossSystemID
 20. Adjuster
 21. BankAccount
 22. LitigationEvent
 23. InjuryRecord
 24. FraudRing
-

6. Claim Versioning Requirements

Claims must support:

- Status changes
- Reserve changes
- Payment lifecycle
- Litigation lifecycle
- Coverage splits
- Reopened claims
- Adjuster reassignment
- SIU referrals

Example:

```
(c:Claim {claim_id:'CLM-1001'})
(v1:ClaimVersion {status:'FNOL'})
(v2:ClaimVersion {status:'INVESTIGATION'})
(v3:ClaimVersion {status:'CLOSED'})
```

```
(c)-[:HAS_VERSION]->(v1)
(c)-[:HAS_VERSION]->(v2)
(c)-[:HAS_VERSION]->(v3)
```

7. Fraud Pattern Categories

Category 1 — Medical Fraud

- Medical mills
- CPT upcoding
- Phantom treatment
- Duplicate billing
- Unbundling

Category 2 — Property Fraud

- Arson rings
- Flood fraud
- Contractor collusion
- Contents inflation

Category 3 — Temporal Fraud

- Claim velocity
- Impossible timelines
- Reserve escalation anomalies

- Policy lapse/reinstate cycles

Category 4 — Identity Fraud

- Synthetic identities
- Identity takeover
- Phantom passengers
- Address fraud

Category 5 — Insider Fraud

- Adjuster/shop collusion
- Adjuster/attorney pipelines
- Phantom payments
- Reopened-claim abuse

Category 6 — Cross-Carrier Fraud

- Same incident across carriers
 - Serial claimants
 - Cross-carrier provider abuse
 - Cross-carrier medical mills
-

8. Privacy & Compliance

PII Handling Rules

Field	Handling
Full SSN	Never store
SSN Last4	Hash
Email	Hash
Bank Account	Last4 only
DOB	Hash/full DOB optional
Address	ZIP+4 preferred

Compliance Goals

- GDPR-aware
- CCPA-aware
- Minimal PII exposure
- Least privilege principles
- Audit-ready configuration

9. Duplicate Prevention Requirements

The developer MUST:

- Use MERGE for all nodes
 - Use MERGE for all relationships
 - Use unique constraints
 - Deduplicate before ingestion
 - Normalize phones/addresses
 - Use entity resolution before graph loading
-

10. Null & Default Handling

Required Protections

Generator Layer

- Every field requires a default value

Loader Layer

- Use COALESCE for nullable properties

Schema Layer

- Constraints on required keys

Example:

```
SET p.phone = COALESCE($phone, 'UNKNOWN')
```

11. GitHub & Version Control Strategy

11.1 Source Control Platform

The project shall use GitHub as the primary source control and collaboration platform.

The architecture and repository structure must support:

- Public or private repository operation
- Future open-source publication

- Enterprise portfolio presentation
- Version-controlled fraud scoring
- CI/CD readiness
- Infrastructure-as-code readiness

11.2 GitHub Usage Requirements

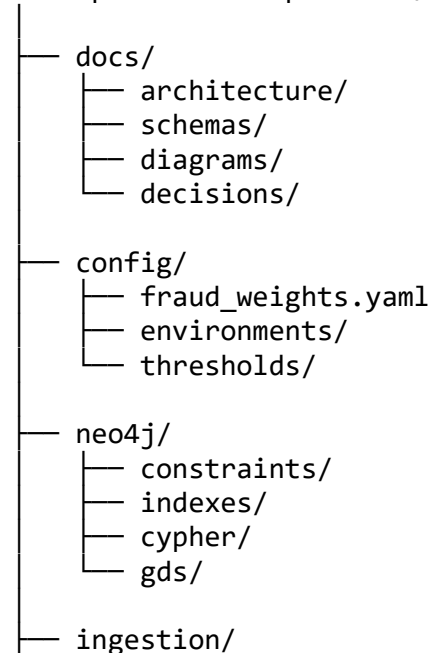
GitHub shall be used for:

Purpose	Usage
Source Control	All application code
YAML Versioning	Fraud scoring configuration
Documentation	Architecture/design docs
Release Management	Tagged releases
Branching	Feature and release branches
CI/CD	Future GitHub Actions integration
Auditability	Weight/configuration history
Collaboration	Pull requests and reviews

11.3 Repository Structure

Recommended repository structure:

enterprise-fraud-platform/



```
|— entity_resolution/  
|— intelligence_engine/  
|— dashboard/  
|— multimodal_ai/  
|— synthetic_data_generator/  
|— tests/  
|— scripts/  
|— deployment/
```

11.4 YAML Configuration Governance

The master YAML fraud scoring configuration file shall:

- Be stored in GitHub
- Be version-controlled
- Be human-readable
- Support rollback
- Support auditing
- Support branch-based experimentation

Example:

```
auto:  
  staged_accident_weight: 35  
  duplicate_claim_weight: 20
```

```
workers_comp:  
  ime_collusion_weight: 45
```

Benefits of GitHub-Based YAML Governance

Benefit	Description
Auditability	Every score change tracked
Rollback	Restore prior scoring versions
Experimentation	Branch-specific tuning
Compliance	Reviewer approval workflows
Collaboration	Analyst + developer coordination

11.5 Git Branching Strategy

Recommended:

Branch	Purpose
--------	---------

Branch	Purpose
main	Stable production-ready code
develop	Active integration branch
feature/*	New capabilities
experiment/*	Fraud scoring experiments
release/*	Release preparation

11.6 Open Source Readiness

The solution should be designed for future public GitHub publication.

Developer must:

- Avoid committing secrets
 - Use environment variables
 - Externalize configuration
 - Use .gitignore properly
 - Avoid proprietary dependencies
 - Document setup procedures
 - Maintain modular architecture
-

11.7 GitHub Actions (Future)

Future CI/CD capabilities may include:

- Automated testing
- YAML validation
- Neo4j schema validation
- Security scanning
- Linting
- Package builds
- Docker builds

This is future-state and not mandatory in Phase 1.

12. Neo4j Desktop Technical Requirements

Desktop Plugins

Required:

- APOC
- GDS

Memory Tuning

Suggested initial settings:

Heap Size: 4G

Page Cache: 4G

Required Practices

- Avoid large Cartesian joins
 - Use indexed lookups
 - Use batch loading
 - Use periodic commits
 - Use profiling on heavy queries
-

13. Development Phases

Phase A — Schema

Deliverables:

- Constraints
- Indexes
- Relationship definitions
- Base Cypher scripts

Phase B — Data Generator

Deliverables:

- Multi-source simulation
- Fraud pattern simulation
- Hartford geography modeling
- ISO simulation

Phase C — Entity Resolution

Deliverables:

- Cross-system mapping
- Fuzzy matching
- Master IDs

Phase D — Intelligence Engine

Deliverables:

- GDS algorithms
- Fraud scoring
- YAML scoring configuration
- Payment hold logic

Phase E — Multi-Modal AI

Deliverables:

- OCR pipeline
- CV analysis
- Image forensics
- Synthetic identity detection

Phase F — Agentic Co-pilot

Deliverables:

- Evidence assembly
- AI narrative generation
- SIU workflow support

Phase G — Dashboard

Deliverables:

- Graph dashboard
 - Timeline dashboard
 - Geospatial dashboard
 - Investigator work queue
-

14. Final Architecture Principles

The solution must be:

- Config-driven
 - Modular
 - API-first
 - Enterprise-inspired
 - GDPR-conscious
 - Market-ready in future phases
 - Neo4j Desktop compatible
 - Migration-ready
 - Explainable
 - Extensible
-

15. Critical Non-Functional Requirements

Performance

- Fraud scoring target < 500ms
- Batch loading optimized
- Efficient graph traversal

Security

- Minimal PII
- Hashed sensitive identifiers
- Config auditability

Maintainability

- No hardcoded weights
- YAML-driven scoring
- Modular architecture

Reliability

- Idempotent ingestion
 - Replay-safe processing
 - Deterministic graph loading
-

16. Developer Deliverables

The implementation team must provide:

1. Neo4j schema scripts
2. Constraints/index scripts
3. Data generator
4. ETL pipeline
5. Fraud scoring engine
6. YAML configuration system
7. GDS analysis queries
8. Dashboard UI
9. OCR pipeline
10. CV pipeline hooks
11. Documentation
12. Setup scripts
13. Sample datasets
14. Test cases
15. Deployment guide

END OF SPECIFICATION